

LAPORAN PROYEK UAS
PEMROGRAMAN BERORIENTASI OBJEK
SISTEM KASIR MERCHANDISE ANIME
(IMPLEMENTASI POLA MVC/DAO MENGGUNAKAN JAVA SWING DAN MYSQL)

Dosen : Saiful Nur Budiman, M.Kom



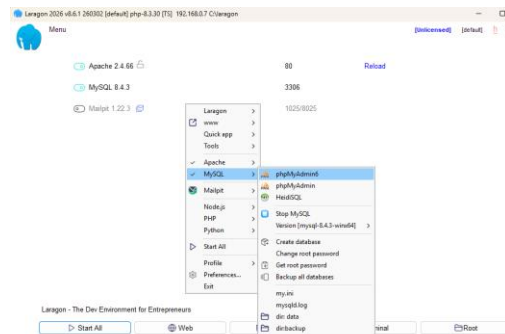
- | | |
|------------------------------------|---------------|
| 1. Tegar Ludwinio Mahardika | (25104410001) |
| 2. Naufal Aqbil Hanif | (25104410003) |
| 3. Muhammad Johan Putra Pratama | (25104410007) |
| 4. Gerry Atha Himawan | (25104410012) |
| 5. Farrel Risqullah Mahardika | (25104410017) |
| 6. Ryz Nuzula Abdillah Nafis Azhar | (25104410018) |
| 7. Syahreza Faiz Pratama | (25104410093) |

PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS TEKNIK DAN INFORMATIKA
UNIVERSITAS ISLAM BALITAR

SEMESTER GENAP 2025/2026

1. Cara Menjalankan Aplikasi

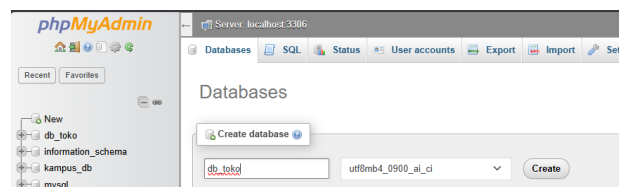
A. Buka Laragon atau XAMPP



Gambar 1.1 Aplikasi Laragon

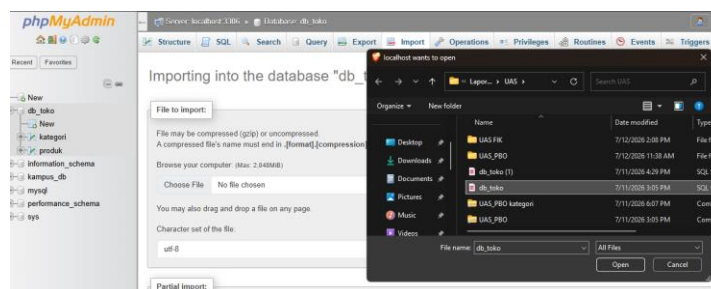
Disini kita menggunakan laragon untuk terhubung ke localhost kami. Langkah pertama hidupkan *Apache* dan *MySQL* agar dapat menghubungkan ke localhost kita. Jika sudah klik kanan pada bagian kosong lalu cari *MySQL* dan klik *phpMyAdmin6* untuk membuka localhost di browser.

B. Konfigurasi phpMyAdmin



Gambar 1.2 phpMyAdmin

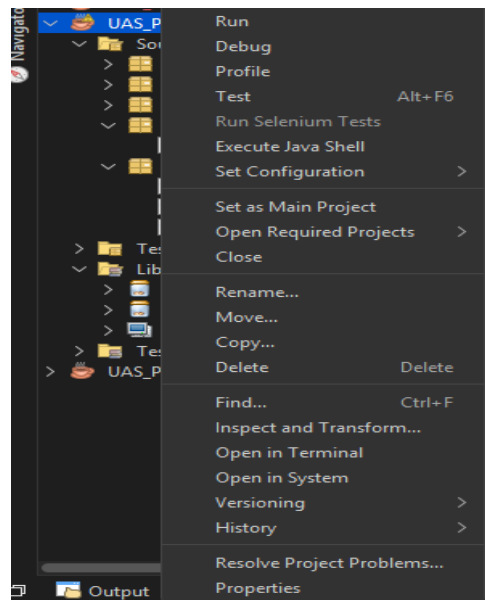
Setelah masuk ke halaman phpMyAdmin import terlebih dahulu data yang sudah di buat dengan cara, buat terlebih dahulu data base kosong dengan nama *db_toko*, setelah itu masuk ke data tersebut dan klik import.



Gambar 1.3 Import Data

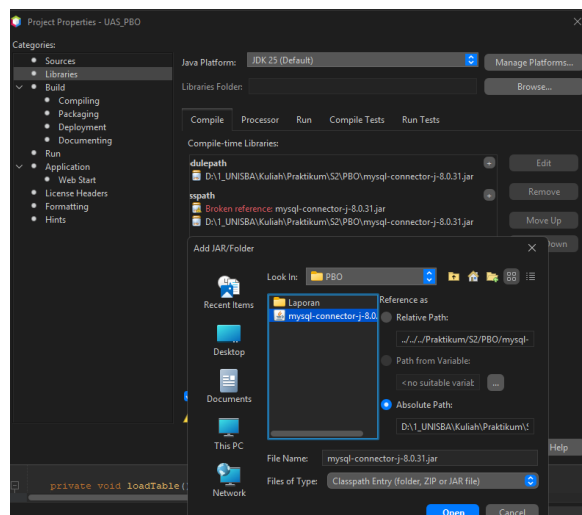
Selanjutnya pilih data yang ingin di import, disini data kita bernama *db_toko* pilih klik open lalu klik import.

C. Buka Netbeans



Gambar 1.4 Project Properties

Sebelum menjalankan program tambahkan terlebih dahulu connector-jar dengan cara klik kanan pada project cari properties untuk menambahkan connector-jar.



Gambar 1.5 Menambahkan File Connector-jar

Setelah klik properties pilih Libraries dan cari tombol “+” untuk memilih connector-jar, setelah itu pilih file dan klik open lalu klik oke untuk menambahkan connector-jar. Fungsi untuk menterjemahkan bahasa dari data base ke program java yang kami buat, singkatnya sebagai jembatan penghubung antara program java dengan data base MySQL agar dapat terbaca oleh program java.

2. Penggunaan Inheritance dan Interface

Dalam pengembangan program Merchandise Anime, prinsip dasar Pemrograman Berorientasi Objek (PBO) berupa Inheritance (pewarisan) dan Interface (antarmuka) diterapkan secara ekstensif melalui pemanfaatan pustaka antarmuka grafis Java Swing. Berikut adalah rincian penerapannya:

A. Penerapan Inheritance (Pewarisan)

Penerapan konsep pewarisan atau *Inheritance* terlihat sangat jelas pada pembuatan kerangka jendela utama aplikasi. Kelas `FormProduk` dan `FormKategori` dibangun dengan mewarisi sifat serta fungsionalitas dari kelas induknya, yang secara eksplisit dideklarasikan dengan sintaks `public class FormProduk extends javax.swing.JFrame` dan `public class FormKategori extends javax.swing.JFrame`. Melalui proses pewarisan komponen `JFrame` ini, kedua form tersebut secara otomatis memiliki kapabilitas standar dari sebuah jendela aplikasi, seperti kemampuan menampung tata letak komponen serta fungsionalitas bawaan window. Sebagai bukti pemanfaatan sifat warisan tersebut, aplikasi dikonfigurasi agar langsung tampil dalam mode layar penuh sesaat setelah dibuka melalui pemanggilan metode turunan `this.setExtendedState(javax.swing.JFrame.MAXIMIZED_BOTH);` yang diletakkan di dalam konstruktor masing-masing kelas.

```
11  /*
12  public class FormProduk extends javax.swing.JFrame {
13
```

Gambar 2.1 Implementasi pewarisan kelas JFrame pada kelas FormProduk.

Selain pengelolaan jendela utama, konsep pewarisan juga diterapkan secara khusus dalam pembuatan form berwujud pop-up yang diimplementasikan pada kelas `DialogTambahProduk`. Berbeda dengan form sebelumnya, kelas ini mewarisi fungsionalitas dari kelas induk `JDialog`, yang ditandai dengan sintaks `public class DialogTambahProduk extends javax.swing.JDialog`. Keunggulan utama dari turunan kelas `JDialog` ini adalah kemampuannya untuk diatur memiliki sifat modal, yang diinisialisasi melalui pemanggilan metode `super(parent, modal)` di dalam blok konstruktor. Sifat modal ini sangat krusial dalam interaksi antarmuka karena ia akan secara otomatis mengunci jendela utama dan mencegah interaksi di luar pop-up hingga proses pengisian data atau aksi di dalamnya selesai dieksekusi.

```

public class DialogTambahProduk extends javax.swing.JDialog {

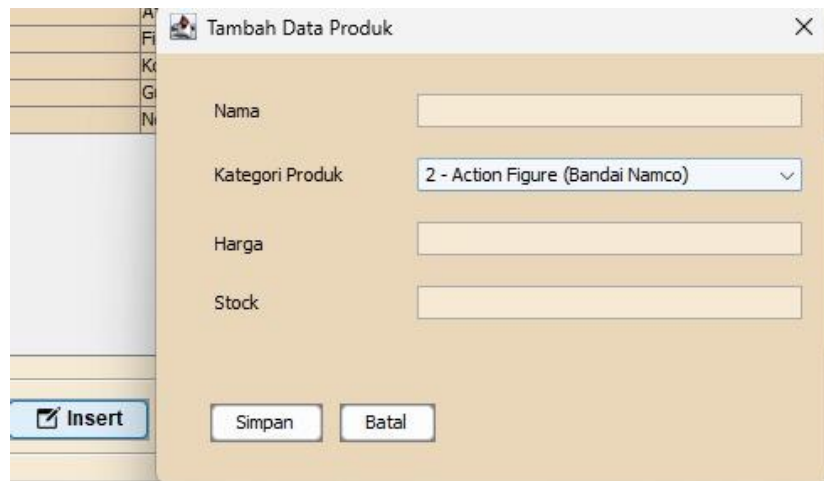
    private dao.ProdukDAO produkDAO = new dao.ProdukDAO();
    private dao.KategoriDAO kategoriDAO = new dao.KategoriDAO();
    private int kodeProdukEdit = 0; // 0 artinya Tambah Data. Jika > 0 artinya Edit Data.

    private static final java.util.logging.Logger logger = java.util.logging.Logger.getLogger(DialogTambahProduk.class.getName());

    /**
     * Creates new form DialogTambahProduk
     */
    public DialogTambahProduk(java.awt.Frame parent, boolean modal) {
        super(parent, modal);
        initComponents();
        setLocationRelativeTo(parent); // Biar pop-up muncul di tengah form produk
        loadKategori(); // Fungsi ini agar ComboBox terisi saat form terbuka!
    }
}

```

Gambar 2.2 Implementasi pewarisan kelas JDialog dan penggunaan sifat modal



Gambar 2.3 Antarmuka jendela pop-up (JDialog) yang mengunci jendela utama (JFrame) di latar belakang.

B. Penerapan Interface (Antarmuka)

Konsep Interface atau antarmuka difokuskan pada aspek interaktivitas aplikasi, khususnya dalam menangani respons program terhadap tindakan pengguna seperti penekanan tombol. Agar sistem dapat mendeteksi dan menjalankan perintah tertentu saat sebuah tombol diklik, komponen tombol tersebut harus mematuhi kontrak interaksi yang disediakan oleh antarmuka ActionListener. Pada aplikasi ini, implementasi antarmuka tersebut tidak diletakkan pada deklarasi class secara tradisional, melainkan langsung disematkan pada setiap komponen menggunakan teknik Method Reference. Sebagai contoh nyata, pada file FormProduk, tombol penambahan data diaktifkan dengan mengikatkan interface tersebut melalui baris kode `btnInsert.addActionListener(this::btnInsertActionPerformed);`

```

123 btnInsert.setBackground(new java.awt.Color(162, 165, 149));
124 btnInsert.setFont(new java.awt.Font("Arial", 1, 12)); // NOI18N
125 btnInsert.setForeground(new java.awt.Color(0, 0, 0));
126 btnInsert.setIcon(new javax.swing.ImageIcon(getClass().getResource("/image/icons8-edit-16.png"))); // NOI18N
127 btnInsert.setMnemonic('E');
128 btnInsert.setText("Insert");
129 btnInsert.setCursor(new java.awt.Cursor(java.awt.Cursor.HAND_CURSOR));
130 btnInsert.addActionListener(this::btnInsertActionPerformed);
131
132 btnUpdate.setBackground(new java.awt.Color(162, 165, 149));
133 btnUpdate.setFont(new java.awt.Font("Arial", 1, 12)); // NOI18N
134 btnUpdate.setForeground(new java.awt.Color(0, 0, 0));

```

Gambar 2.4 Pengikatan antarmuka ActionListener pada komponen tombol menggunakan Method Reference.

Pendekatan implementasi antarmuka yang seragam juga diterapkan pada berbagai jendela lainnya di dalam aplikasi. Pada jendela pop-up DialogTambahProduk, tombol untuk memproses data baru menggunakan sintaks. untuk mendeteksi aksi dari pengguna. Melalui mekanisme antarmuka ActionListener ini, setiap kali klik terjadi, sistem akan langsung memicu event dan mengeksekusi logika pemrograman seperti memvalidasi teks hingga menyimpan data ke basis data yang tertanam di dalam metode spesifik tersebut.

```

108 private void btnSimpanActionPerformed(java.awt.event.ActionEvent e) {
109     try {
110         if (txtNama.getText().isEmpty() || txtKategori.getText().isEmpty() || txtStok.getText().isEmpty()) {
111             javax.swing.JOptionPane.showMessageDialog(this, "Forma belum benar diisi!");
112             return;
113         }
114         Model.Produk p = new Model.Produk();
115         p.setNama(txtNama.getText());
116         p.setKategori(Integer.parseInt(txtKategori.getText()));
117         p.setStok(Integer.parseInt(txtStok.getText()));
118         // ambil objek kategori yang dipilih user dari ComboBox
119         Model.Kategori k = (Model.Kategori) cmbKategori.getSelectedItem();
120         p.setKode_produk(k.getKode());
121         if (kodeProdukBaru == 0) {
122             // MODE TAMBAH
123             if (produkDAO.insert(p)) {
124                 javax.swing.JOptionPane.showMessageDialog(this, "Produk Berhasil Ditambahkan!");
125                 this.dispose(); // Tutup pop-up
126             }
127         } else {
128             // MODE EDIT
129             p.setKode_produk(produkDAO.getId());
130             if (produkDAO.update(p)) {
131                 javax.swing.JOptionPane.showMessageDialog(this, "Produk Berhasil Diupdate!");
132                 this.dispose(); // Tutup pop-up
133             }
134         }
135     } catch (NumberFormatException e) {
136         javax.swing.JOptionPane.showMessageDialog(this, "Harga dan Stok wajib diisi dengan Angka!", "Error Input", javax.swing.JOptionPane.ERROR_MESSAGE);
137     }
138 }

```

Gambar 2.5 Metode yang dieksekusi sebagai respons dari event ActionListener saat tombol ditekan.

3. Pola Dao Yang Digunakan

A. Penerapan Pola MVC (Model – DAO – View)

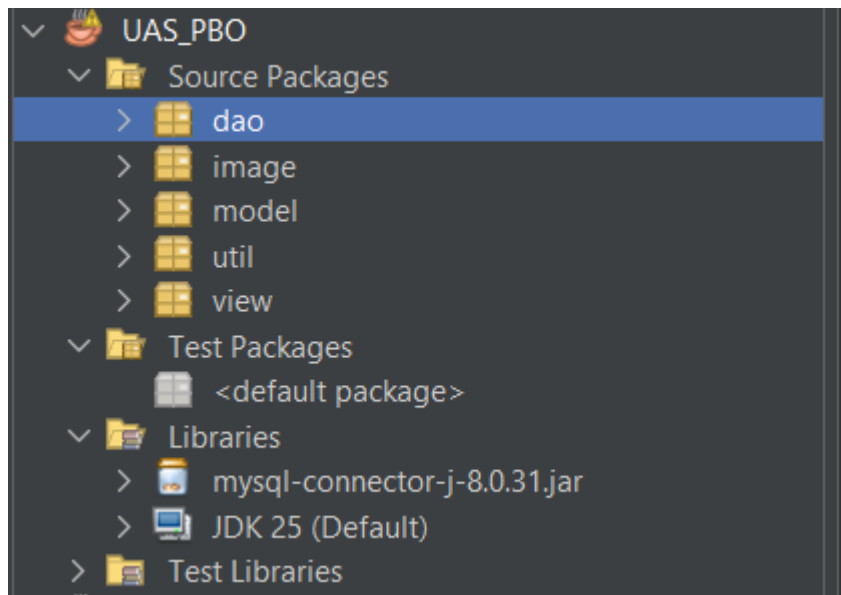
1. Konsep Pola MVC/DAO

Pada aplikasi Kasir Merchandise Anime, struktur program dibagi ke dalam tiga package utama, yaitu Model, DAO (Data Access Object), dan View. Pembagian package ini bertujuan agar setiap bagian program memiliki tanggung jawab yang berbeda sehingga kode menjadi lebih terstruktur, mudah dipahami, dan mudah dikembangkan.

Setiap package memiliki fungsi sebagai berikut.

- Model digunakan untuk menyimpan data aplikasi dalam bentuk objek.
- DAO digunakan untuk menghubungkan aplikasi dengan database serta menjalankan seluruh perintah SQL.
- View digunakan sebagai antarmuka (User Interface) yang berinteraksi langsung dengan pengguna.

Dengan pembagian tersebut, perubahan pada tampilan tidak akan memengaruhi proses database, begitu pula perubahan database tidak akan mengganggu tampilan aplikasi.



Gambar 3.1 menunjukkan struktur package pada aplikasi Kasir Merchandise Anime yang terdiri dari package Model, DAO, dan View sebagai implementasi pola MVC.

2. Package Model

Package Model berfungsi sebagai representasi data yang terdapat pada database. Pada project ini package Model terdiri dari class seperti Produk dan Kategori yang digunakan untuk menyimpan data sebelum diproses lebih lanjut oleh DAO.

Seluruh atribut pada class Model menggunakan modifier private sehingga data tidak dapat diakses secara langsung dari luar class. Untuk mengakses maupun mengubah nilai atribut digunakan method getter dan setter. Penerapan ini merupakan konsep Encapsulation dalam pemrograman berorientasi objek yang bertujuan menjaga keamanan dan konsistensi data.

Objek Model berperan sebagai media pertukaran data antara package View dengan package DAO. Ketika pengguna mengisi data pada form, nilai tersebut terlebih dahulu disimpan ke dalam objek Model, kemudian objek tersebut dikirim ke DAO untuk diproses ke database.

Fungsi package Model antara lain:

- Menyimpan data produk dan kategori.
- Menerapkan variabel private beserta getter dan setter.
- Menjadi objek pertukaran data antara View dan DAO.
- Merepresentasikan struktur tabel pada database.

```

8 public class Produk {
9     private int kode;
10    private int kode_produk;
11    private String nama;
12    private int harga;
13    private int stock;
14
15    public int getKode() { return kode; }
16    public void setKode(int kode) { this.kode = kode; }
17
18    public int getKode_produk() { return kode_produk; }
19    public void setKode_produk(int kode_produk) { this.kode_produk = kode_produk; }
20
21    public String getNama() { return nama; }
22    public void setNama(String nama) { this.nama = nama; }
23
24    public int getHarga() { return harga; }
25    public void setHarga(int harga) { this.harga = harga; }
26
27    public int getStock() { return stock; }
28    public void setStock(int stock) { this.stock = stock; }
29 }

```

Gambar 3.2 menunjukkan class Produk yang digunakan sebagai representasi data produk. Seluruh atribut menggunakan modifier private sehingga hanya dapat diakses melalui method getter dan setter.

```

8 public class Kategori {
9     private int kode;
10    private String kategori;
11    private String brand;
12
13    public int getKode() { return kode; }
14    public void setKode(int kode) { this.kode = kode; }
15
16    public String getKategori() { return kategori; }
17    public void setKategori(String kategori) { this.kategori = kategori; }
18
19    public String getBrand() { return brand; }
20    public void setBrand(String brand) { this.brand = brand; }
21
22    // Format khusus agar tampil rapi di dropdown (JComboBox)
23    @Override
24    public String toString() {
25        return kode + " - " + kategori + " (" + brand + ")";
26    }
27 }

```

Gambar 3.3 menunjukkan class Kategori yang digunakan untuk menyimpan informasi kategori merchandise beserta atribut-atribut yang dimilikinya.

3. Package DAO (Data Access Object)

Package DAO (Data Access Object) berfungsi sebagai penghubung antara aplikasi dengan database. Seluruh proses CRUD (Create, Read, Update, Delete) dan eksekusi perintah SQL ditempatkan pada package ini sehingga package lain tidak berhubungan langsung dengan database.

Pada aplikasi ini terdapat dua class DAO, yaitu KategoriDAO dan ProdukDAO. Kedua class tersebut bertugas mengelola data kategori dan produk, mulai dari mengambil data,

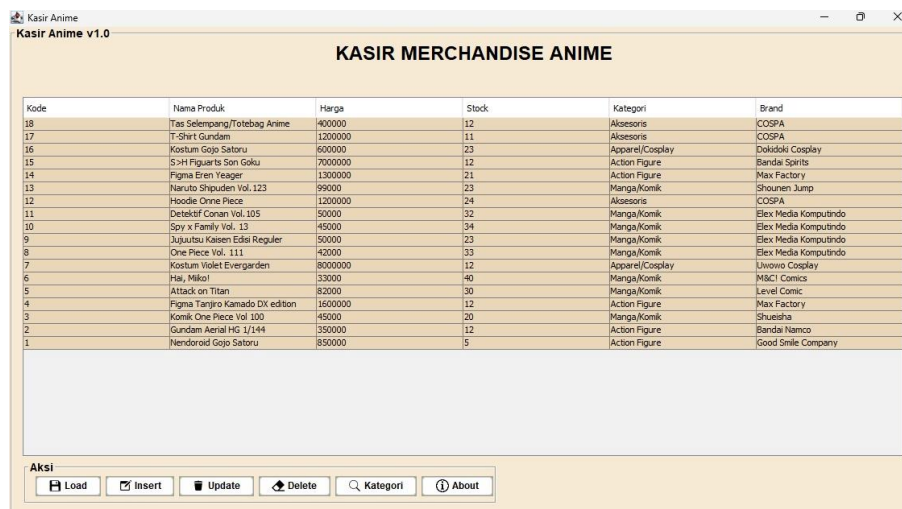
menambahkan data, memperbarui data, hingga menghapus data dari database. Dengan adanya package DAO, seluruh logika akses database dipusatkan dalam satu tempat sehingga kode program menjadi lebih terstruktur dan mudah dipelihara.

Dengan adanya package DAO, seluruh proses CRUD dan komunikasi dengan database dipusatkan pada satu tempat sehingga kode program menjadi lebih rapi, mudah dipelihara, dan lebih mudah dikembangkan apabila terjadi perubahan struktur database.

4. Package View

Package View merupakan bagian yang menangani seluruh tampilan antarmuka (User Interface) pada aplikasi. Package ini berisi class FormProduk, FormKategori, dan DialogTambahProduk yang dibuat menggunakan Java Swing. Pada package ini pengguna dapat melihat data, memasukkan data baru, mengubah data, maupun menghapus data melalui komponen seperti JFrame, JDialog, JTable, JButton, JTextField, dan JComboBox.

Class FormProduk berfungsi sebagai halaman utama aplikasi yang menampilkan seluruh data produk ke dalam tabel serta menyediakan tombol Load, Insert, Update, Delete, dan Kategori. Data produk ditampilkan melalui method loadTable() yang mengambil data dari ProdukDAO. Setelah pengguna melakukan proses tambah, ubah, maupun hapus data, tabel akan dimuat kembali sehingga informasi yang tampil selalu diperbarui.



Kode	Nama Produk	Harga	Stock	Kategori	Brand
18	Tas Selempang/Totebag Anime	400000	12	Aksesoris	COSPA
17	T-Shirt Gundam	1200000	11	Aksesoris	COSPA
16	Kostum Gojo Saturu	600000	23	Apparel/Cosplay	Dolaki Cosplay
15	S-Hi Figure Son Goku	7000000	12	Action Figure	Bandai Spirits
14	Figure Eren Yeager	1300000	21	Action Figure	Max Factory
13	Naruto Shippuden Vol. 123	99000	23	Manga/Komik	Shounen Jump
12	Hoodie Onne Piece	1200000	24	Aksesoris	COSPA
11	Detectif Conan Vol. 105	50000	32	Manga/Komik	Elex Media Komputindo
10	Spy x Family Vol. 13	45000	34	Manga/Komik	Elex Media Komputindo
9	Jujutsu Kaisen Edisi Regular	50000	23	Manga/Komik	Elex Media Komputindo
8	One Piece Vol. 111	42000	33	Manga/Komik	Elex Media Komputindo
7	Kostum Violet Evergarden	8000000	12	Apparel/Cosplay	Uworo Cosplay
6	Hai, Mikai	13000	40	Manga/Komik	M&C Comics
5	Attack on Titan	82000	30	Manga/Komik	Level Comic
4	Figure Tanjiro Kamado DX edition	1600000	12	Action Figure	Max Factory
3	Komik One Piece Vol 100	45000	20	Manga/Komik	Shueisha
2	Gundam Aerial HG 1/144	350000	12	Action Figure	Bandai Namco
1	Nendoroid Gojo Saturu	850000	5	Action Figure	Good Smile Company

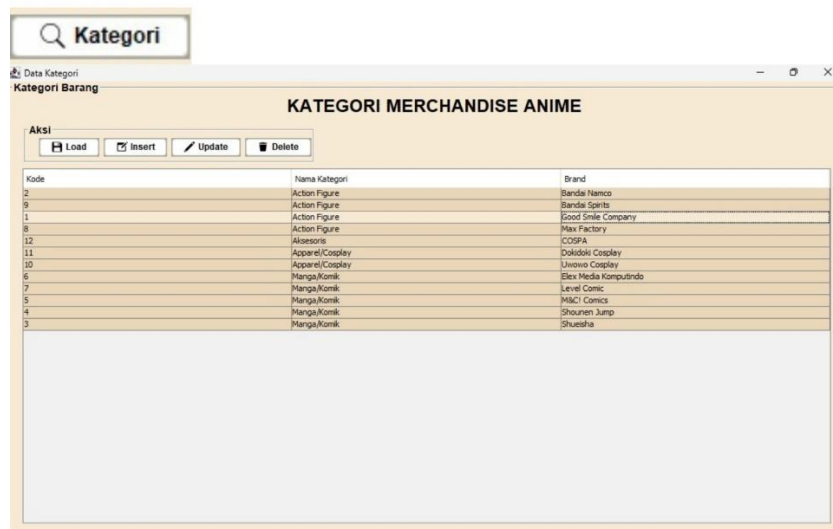
Aksi

Load Insert Update Delete Kategori About

Gambar 3.5 menunjukkan halaman utama aplikasi yang digunakan untuk menampilkan dan mengelola data produk.

Class FormKategori digunakan untuk mengelola data kategori merchandise. Form ini menampilkan daftar kategori dalam JTable dan menyediakan fitur untuk menambah, memperbarui, serta menghapus kategori melalui KategoriDAO. Dengan demikian, seluruh

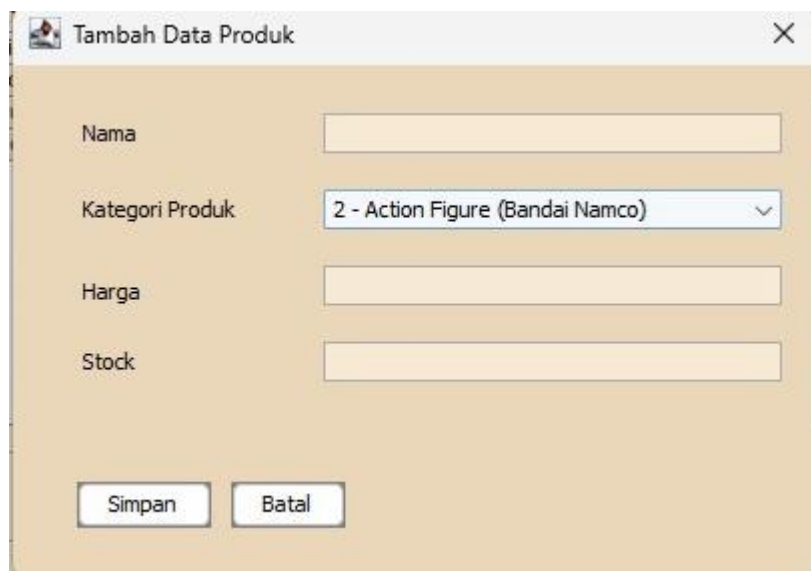
proses pengolahan data tetap dipisahkan dari tampilan aplikasi.



Kode	Nama Kategori	Brand
2	Action Figure	Bandai Namco
9	Action Figure	Bandai Spirits
1	Action Figure	Good Smile Company
8	Action Figure	Max Factory
12	Accessories	COSPA
11	Apparel/Cosplay	Dokidoki Cosplay
10	Apparel/Cosplay	Universe Cosplay
6	Manga/Komik	Elex Media Komputindo
7	Manga/Komik	Level Comic
5	Manga/Komik	MISC Comics
4	Manga/Komik	Shounen Jump
3	Manga/Komik	Shueisha

Gambar 3.6 menunjukkan tampilan form kategori yang digunakan untuk mengelola data kategori merchandise.

Class DialogTambahProduk merupakan form dialog (popup) yang digunakan untuk proses penambahan maupun pembaruan data produk. Dialog ini mengambil daftar kategori dari KategoriDAO, menampilkan pilihan kategori pada JComboBox, kemudian membuat objek Produk berdasarkan input pengguna sebelum mengirimkannya ke ProdukDAO melalui method insert() atau update().

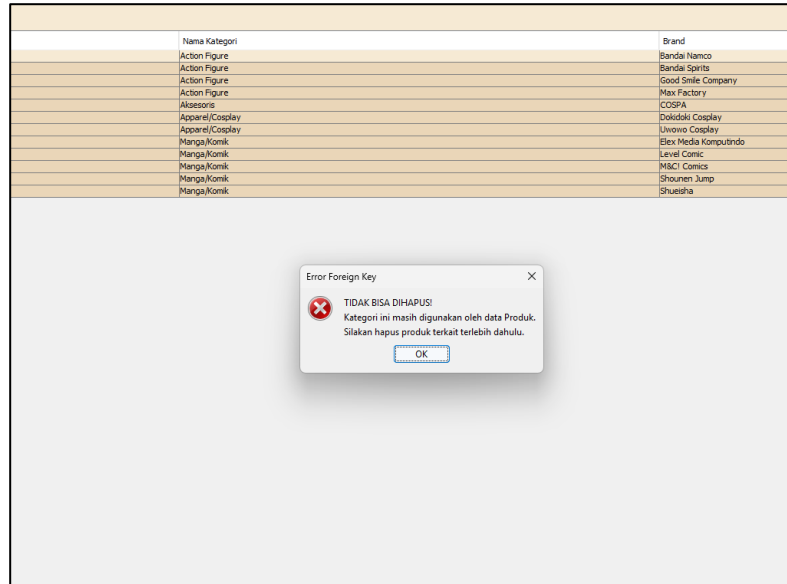


Gambar 3.7 menunjukkan dialog penambahan data produk yang digunakan sebagai media input data sebelum diproses ke database.

Dengan demikian, package View hanya bertugas mengelola tampilan dan interaksi pengguna tanpa menjalankan perintah SQL secara langsung.

4. Penanganan Error Foreign Key

A. Penanganan Kesalahan (Error Handling)



Gambar 4.1 Screenshot Error Foreign Key

```
96 // 4. DELETE: Menghapus kategori (Lengkap dengan Cegatan Foreign Key)
97 public boolean delete(int kode) {
98     String sql = "DELETE FROM kategori WHERE kode = ?";
99     try {
100         PreparedStatement ps = conn.prepareStatement(sql);
101         ps.setInt(1, kode);
102         ps.executeUpdate();
103         return true;
104     } catch (SQLException e) {
105         // Error code 1451 adalah aturan RESTRICT dari database jika data masih dipakai di tabel produk
106         if (e.getErrorCode() == 1451) {
107             JOptionPane.showMessageDialog(null, "TIDAK BISA DIHAPUS!\nKategori ini masih digunakan oleh data Produk.");
108         } else {
109             JOptionPane.showMessageDialog(null, "Error Delete: " + e.getMessage());
110         }
111         return false;
112     }
113 }
114 }
```

Gambar 4.2 Screenshot Catch blok try-catch (SQLException)

Sistem kasir ini telah dilengkapi dengan mekanisme error handling untuk menjaga integritas data pada database. Tabel Kategori memiliki hubungan Foreign Key dengan tabel Produk, sehingga suatu kategori yang masih digunakan oleh produk tidak dapat dihapus secara langsung.

Berdasarkan gambar di atas, pengguna mencoba menghapus salah satu data kategori yang terdapat pada tabel kategori. Namun, setelah tombol hapus dijalankan, sistem tidak langsung menghapus data tersebut, melainkan menampilkan sebuah kotak dialog dengan

judul "Error Foreign Key" yang berisi pesan "TIDAK BISA DIHAPUS! Kategori ini masih digunakan oleh data Produk. Silakan hapus produk terkait terlebih dahulu."

Pesan tersebut memberikan informasi kepada pengguna bahwa kategori yang dipilih masih memiliki hubungan dengan data pada tabel produk, sehingga proses penghapusan tidak dapat dilakukan. Hal ini bertujuan untuk mencegah terjadinya data yang tidak konsisten, misalnya produk yang masih menggunakan kategori tertentu tetapi kategori tersebut sudah tidak ada di dalam database. Oleh karena itu, sistem secara otomatis membatalkan proses penghapusan dan mempertahankan data kategori agar hubungan antar data tetap terjaga.

Dengan adanya pesan peringatan tersebut, pengguna dapat mengetahui secara jelas alasan mengapa proses penghapusan gagal. Selain itu, sistem juga memberikan petunjuk kepada pengguna untuk terlebih dahulu menghapus atau mengubah data produk yang masih menggunakan kategori tersebut sebelum mencoba menghapus kategori kembali. Mekanisme ini membuat proses pengelolaan data menjadi lebih aman, mengurangi kemungkinan kesalahan pengguna, serta membantu menjaga keakuratan dan integritas data yang tersimpan di dalam sistem kasir.